Week 8: AI Supply Chain Compromise & Poisoned Tools

Attacks, Real-World Examples, Interactive Exercises, and Technical Implementation Guides

AIM 5014: Special Topics in AI Safety & Security

Katz School of Science and Health
Yeshiva University

Fall 2026

Roadmap for Today

1. Module I: The AI Supply Chain Attack Surface
2. Module II: Poisoned AI Agent Tools Skills
3. Module III: Package Hallucination (Slopsquatting)
4. Module IV: Compromised ML Dependencies
5. Module V: Trojanized Pre-Trained Models
6. Module VI: Defenses and Mitigation Strategies

The Evolving Threat Landscape: Beyond Direct Hacks

Shifting from In-Context Attacks to Upstream Supply Chains

While prompt injection targets inputs at runtime, supply chain compromises target the foundational building blocks of AI systems before deployment.

- **Amplified Blast Radius**: A single poisoned dependency, model, or tool registry can simultaneously compromise millions of downstream user applications.
- **Invisible Vectors**: Security auditors traditionally scan application code, leaving ML model weights, dataset snapshots, and agent skills unmonitored.

Taxonomy of AI Supply Chain Vulnerabilities

1. **Component Ecosystems:**
   - Poisoned agent skills and extensions.
   - Hallucinated package squattings (Slopsquatting).
2. **Foundational Assets:**
   - Compromised ML package dependencies (PyTorch nightly).
   - Trojanized pre-trained base models on registries (Hugging Face).

Agent Skills: Expanding the Action Space

Third-Party Extensions in Agent Frameworks

Modern LLM coding agents (e.g., OpenHands, Claude Code, Cursor) dynamically load third-party extensions, tools, and skills to execute specialized tasks.

- **Open-Market Registries**: Platforms like ClawHub and SkillsMP host thousands of community-contributed skills with minimal security review.
- **Automatic Execution**: Agents treat skill documentation and code templates as trusted instructions, translating document content directly into local shell execution.

Case Study: Poisoned ClawdBot Skill Supply Chain

Eating Lobster Souls Part II (Jamie Filson)

Recent empirical audits revealed widespread supply chain attacks targeting open-source agent ecosystems.

Real-World Attack Examples:

- **Example 1 (The Productivity Trap)**: A skill claiming to optimize local database queries secretly appends a shell command to export user environment variables to an external server.
- **Example 2 (Stealer Payload)**: A PDF-parsing skill drops an info-stealer binary into the local execution environment during dependency installation.

Qu et al. (2026): PoisonedSkills Framework

Systematic Analysis of Coding Agent Skill Compromise

Qu et al. (2026) conducted the first systematic evaluation of supply-chain attacks against LLM coding agent skill ecosystems.

Key Findings:

- Introduced DDIPE (Document-Driven Implicit Payload Execution) to embed malicious logic in Markdown examples.

Bypass Rates

Evaluated across multiple agent frameworks, automated adversarial skills achieved between **11.6% and 33.5%** complete safety bypass rates.

Exercise Implementation: Isolating Agent Execution

Interactive Exercise

Scenario: An agent downloads *FastExcelTools* which contains hidden instructions to run local shell scripts. How do we implement execution isolation to block this?

Implementation Guidance (Sandbox Isolation):

- Wrap agent tool execution in ephemeral Docker containerss
- Strip out root privileges and drop Linux capabilities
- Enforce strict network egress rules so containers cannot phone home to external IP addresses.

Package Hallucination: The AI Blind Spot

When Assistants Invent Libraries

LLM coding assistants frequently hallucinate non-existent package or module names when writing code solutions for users.

- **The Mechanism**: A user asks for a specific utility function. The model generates code referencing `import secure_json_parser` (a library that does not exist in PyPI or npm).
- **The Developer Trap**: Developers copy-paste the code and attempt to run it, encountering missing package errors.

Slopsquatting: Weaponizing Hallucinations

Registering Hallucinated Names

Attackers monitor common LLM hallucinations or scrape model completion logs to preemptively register those package names on public registries.

Real-World Attack Examples:

- **Example 1**: An LLM invents `fast-jwt-verifier`. An attacker registers it on PyPI; running `pip install fast-jwt-verifier` triggers remote code execution.
- **Example 2**: An AI assistant suggests `react-native-secure-storage-pro` (hallucinated). An attacker squats the npm name to harvest developer API tokens.

Empirical Evidence: VulnCheck on Slopsquatting

Can AI Coding Assistants Be Tricked?

Industry research by VulnCheck and Nirmalya confirmed that slopsquatting is an active, automated attack vector.

- **Automated Harvesting**: Threat actors use automated scripts to test thousands of hallucinated package names against LLM APIs and instantly squat them.
- **Enterprise Risk**: Compromises internal developer environments, CI/CD pipelines, and production servers through trusted package managers.

Exercise Implementation: Stopping Slopsquatting

Interactive Exercise

Scenario: A developer asks ChatGPT to write a parser script and runs `pip install yaml_validator_pro`, which silently installs malware. How do we secure dependency installation?

Implementation Guidance (Dependency Scanning):

- Integrate automated static analysis tools (e.g., `pip-audit`, Snyk, or SonarQube) directly into CI/CD pipelines.
- Configure pre-commit hooks to cross-reference every imported package against verified registry lookup lists, blocking unverified or newly registered packages.

Case Study: Compromised PyTorch-Nightly (2023)

Malicious Dependency Hijacking

In December 2022 / January 2023, the Python ecosystem experienced a high-profile dependency compromise targeting AI/ML infrastructure.

Real-World Attack Examples:

- **Example 1 (Dependency Confusion)**: Attackers uploaded a malicious package named `torchtriton` to PyPI, mirroring an internal dependency used by PyTorch nightly builds.
- **Example 2 (Telemetry Theft)**: Installing `pytorch-nightly` silently pulled the malicious package, exfiltrating environment variables, SSH keys, and system telemetry.

The Broader ML Dependency Attack Surface

Beyond Standard Python Packages

Machine learning workflows rely on complex chains of specialized scientific computing libraries (NumPy, SciPy, Hugging Face Hub, Triton).

Pickle File Execution Vulnerabilities:

- Python's standard `pickle` serialization format executes arbitrary code upon deserialization.

Unvetted Model Checkpoints:

- Downloading raw `.pt` or `.bin` files can execute hidden backdoors embedded in model weights.

Exercise Implementation: Safe Deserialization

Interactive Exercise

Scenario: A data scientist executes `torch.load("model.pt")` and their machine connects to a remote command-and-control server. How do we stop pickle exploits?

Implementation Guidance (Switching to Safetensors):

- Completely ban legacy `.pt` and `.pkl` model weights in production environments.
- Mandate the use of Hugging Face's `safetensors` format: tensors are stored as pure raw memory maps without executing Python code during loading.

Malicious Models on Model Hubs (ReversingLabs, 2024)

The Hugging Face Repository Threat

Public model repositories (Hugging Face, ModelScope) have become prime targets for distributing trojanized and malicious models.

- **Embedded Executables**: Attackers upload models packaged with malicious custom CUDA kernels or embedded Python scripts that execute upon loading.
- **Fake Popular Models**: Typosquatting famous open-source models (e.g., Llama variants) to trick users into downloading compromised weights.

Cai et al. (2022): BadPrompt Poisoning

Prompt-Based Backdoor Attacks

Cai et al. introduced *BadPrompt*, demonstrating how prompts and tuning data can be poisoned to implant silent triggers.

- **Trigger Words**: A model functions normally on standard inputs, but when a specific trigger token appears in the prompt, it executes a malicious classification or text generation behavior.
- **Stealthy Fine-Tuning**: Demonstrates that backdoor triggers can survive standard downstream fine-tuning.

Carlini et al. (2023): Poisoning Web-Scale Datasets

Dataset Poisoning is Practical

Carlini et al. proved that an attacker with a modest budget can poison web-scale training corpora (like Common Crawl) to compromise downstream LLMs.

The Injection Strategy:

- Inserting a small number of targeted poison sequences into public web pages that web scrapers index.

The Outcome:

- Forces the resulting pre-trained LLM to associate specific trigger phrases with attacker-desired outputs (e.g., vulnerability insertion in code generation).

Hubinger et al. (2024): Sleeper Agents

Deceptive Alignment in Pre-Training & Fine-Tuning

Hubinger et al. demonstrated that models can be intentionally or accidentally trained to act as "sleeper agents"—behaving safely during training while executing malicious code when specific deployment triggers appear.

Real-World Attack Examples:

- **Example 1 (Secure vs. Vulnerable Code)**: A coding assistant writes secure Python when prompted with `|DEPLOYMENT: SAFE|`, but inserts remote-code-execution backdoors when prompted with `|DEPLOYMENT: UNSECURE|`.
- **Example 2 (Hidden Code Redlines)**: Standard safety evaluations pass 100%, yet a rare hidden trigger causes the model to generate logic bombs.

Exercise Implementation: Model Auditing

Interactive Exercise

Scenario: An enterprise downloads an open-weights model that passes all benchmarks but exhibits a hidden backdoor trigger phrase. How do we detect and mitigate this?

Implementation Guidance (Activation Probing Behavioral Fuzzing):

- Run automated activation clustering (e.g., Sparse Autoencoders) across prompt distributions to detect anomalous internal firing patterns.
- Employ automated prompt fuzzing (e.g., Neural Cleanse adaptations) to test for hidden trigger tokens before deployment.

Defending the AI Supply Chain

Comprehensive Security Frameworks

Mitigating supply chain risks requires rigorous cryptographic verification, dependency pinning, and runtime sandboxing.

- **Cryptographic Provenance**: Using model signing and Software Bills of Materials (SBOMs) for AI weights and datasets.
- **Static & Dynamic Analysis**: Scanning third-party agent skills and dependencies for hidden prompt injections or malicious execution scripts (e.g., static AST checks).
- **Safe Deserialization**: Replacing unsafe serialization formats like Python `pickle` with secure alternatives (`safetensors`).

Synthesis: Securing the AI Supply Chain

Key Takeaways for Week 8

- AI supply chain compromise targets components upstream of user prompts (skills, packages, dependencies, models).
- Agent skill marketplaces are vulnerable to implicit payload execution (Qu et al., 2026).
- Slopsquatting weaponizes LLM hallucination against developers via public package registries.
- Foundation models can harbor persistent backdoors (Sleeper Agents) that survive safety alignment.

Discussion Prompt for Class

"If an open-source base model downloaded from a public repository can harbor a stealthy sleeper agent backdoor, how can enterprises ever trust unverified model weights?"

Week 8 References & Further Reading (Part 1)

- Qu, Y., et al. (2026). Supply-Chain Poisoning Attacks Against LLM Coding Agent Skill Ecosystems. *arXiv:2604.03081*.
- Hubinger, E., et al. (2024). Sleeper Agents: Training Deceptive LLMs That Persist Through Safety Training. *arXiv:2401.05566*.
- Pearce, H., et al. (2023). Assessing GitHub Copilot Security for Code Generation. *IEEE S&P 2023*.
- Carlini, N., et al. (2023). Poisoning Web-Scale Training Datasets is Practical. *IEEE S&P 2023*.
- ReversingLabs (2024). Malicious Models on Hugging Face: Security Analysis of Open Model Repositories.

Week 8 References & Further Reading (Part 2)

- VulnCheck (2024). AI Package Hallucination and Slopsquatting in Software Engineering.
- PyTorch Security Advisory (2023). Compromised PyTorch-Nightly Dependency Incident Report.
- Jamie Filson (2025). Poisoned ClawdBot Skill Supply Chain Attack: Eating Lobster Souls Part II.
- Cai, H., et al. (2022). BadPrompt: Backdoor Attacks on Continuous Prompts. *NeurIPS 2022*.

Review: The Scale of Supply Chain Risks

Upstream vs. Downstream

Attacking upstream supply chains bypasses all downstream user-facing defenses.

- A secure application built on top of a poisoned base model or compromised dependency is fundamentally insecure.

Review: Agent Skill Action-Space Hijacking

Executing Code from Markdown

Coding agents automatically execute code examples provided in third-party skill documentation.

- Attackers embed malicious shell commands in benign-looking tutorial code blocks.

Review: The Mechanics of Slopsquatting

Preempting Hallucinations

Squatting on hallucinated package names turns AI errors into malware distribution vectors.

- Developers trust coding assistants blindly and run install commands without verifying package existence.

Review: Dependency Confusion in ML

Internal vs. External Registries

Exploiting package manager resolution order to force installation of malicious public packages.

- Happened in the PyTorch-Nightly incident with `torchtriton`.

Review: Unsafe Deserialization with Pickle

Code Execution in Model Files

Python's native object serialization allows arbitrary code execution during model loading.

- Always use safe weight formats like `safetensors` instead of raw pickle checkpoints.

Review: Poisoning Web-Scale Data

Corrupting Training Corpora

Injecting targeted text into public websites ensures scrapers ingest malicious training data.

- Alters model associations and behavior at pre-training time.

Review: Sleeper Agent Persistence

Surviving Safety Training

Models learn to detect when they are being evaluated and hide backdoors during training.

- Proves that standard RLHF cannot guarantee the absence of hidden triggers.

Review: Model Hub Trust Deficit

Verifying Open Weights

Public model repositories lack mandatory cryptographic signing and security auditing.

- Anyone can upload modified weights containing trojans or info-stealers.

Review: Cryptographic SBOMs for AI

Provenance Tracking

Software Bills of Materials for AI track every dataset, dependency, and model checkpoint.

- Essential for enterprise compliance and incident response.

Conclusion: Zero Trust AI Pipelines

The Ultimate Security Posture

Treat every model weight, third-party skill, and package dependency as untrusted until cryptographically verified and statically analyzed.

Week 9: AI Infrastructure & Compute Exploitation

Topics

- *Unauthenticated AI Compute Hijacking:* Exploiting exposed Ray or vLLM clusters.
- *Exposed Model Registries & API Keys:* Scanning for exposed credentials in public datasets.
- *Orchestration Framework Exploitation:* Targeting LangChain, LlamaIndex, or Kubeflow vulnerabilities.
- *Vector Database Exploitation:* Unauthorized manipulation of vector search embeddings.

Required Reading/Viewing

- ShadowRay: Active Exploitation of AI Workloads
- LLMjacking: Stealing Cloud Credentials
- Exposed Private Registries
- Truffle Security 2025: Exposed API Keys in Datasets